Term Project Deliverable #2: Microservice Design with Collaborations

Project: CampusKit -- peer-to-peer short-term equipment rental for university students

1. Core Business Use Cases & Actors

UC-1: Publish an Equipment Listing (Primary Actor: Owner)
UC-2: Book and Pay for a Rental (Primary Actor: Renter)
UC-3: Complete the Handover, Return, and Settlement (Primary Actors: Owner and Renter)

Supporting capability: S-0 register/login/logout identifies the student through a JWT. S-1 email verification and administrator student-status review establish eligibility before publishing/booking. S-2 records condition evidence; S-3 permits participant reviews only after rental closure.

2. External Systems

Payment Gateway: deferred production integration behind Payment's adapter. The current local ledger models payment outcomes without real money; provider selection remains deferred.
University Identity Provider: deferred. The demo uses email verification and separate administrator review of student status.
Email & Push Providers: deferred. Current delivery is a MongoDB in-app inbox populated from RabbitMQ events.
Object Storage (S3): deferred. Demo photographs are compressed inline data URLs; signed upload/download URLs are a future integration.

3. Service, Operations, and Collaborators

The table maps current capabilities to their owning services; operation labels describe responsibilities rather than exact endpoint names. Seven business services own separate logical databases and credentials. The API Gateway validates and propagates JWTs and routes REST/gRPC requests; data-owning services enforce role, ownership and participation. Scheduled operations are background jobs. Deferred capabilities are listed separately below.

Service & responsibility	Current operations	Collaborators / delivery	Protocol	Database
Student Identity Authentication, verification, eligibility; identity_db	Register/login; verify email; admin student verification; check eligibility/profile	Gateway validates and propagates JWT. Other services query eligibility/profile. Identity writes notification outbox events.	REST	PostgreSQL
Listing Items, pricing, screening; listing_db	Create safe DRAFT; search/detail/quote; moderation; publish/unpublish	Identity: eligibility and payout profile. Booking: calendar setup/availability (gRPC). Review: reputation summary. Outbox events.	REST; gRPC client	PostgreSQL
Booking Authoritative calendar and agreed terms; booking_db	Set calendar; check availability; request/approve/cancel booking; scheduled expiry and compensation	Identity: eligibility. Listing: detail/quote. Payment: authorize/void. Rental: create/cancel. Outbox events.	gRPC; REST clients	PostgreSQL

Rental Custody and settlement workflow; rental_db	Pickup/return evidence and codes; confirm pickup/return; settlement/retry; scheduled overdue/reminders	Booking: status/calendar release and affected bookings (gRPC). Payment: capture/settle. Publishes outbox events. Closes after successful settlement.	REST; gRPC client	PostgreSQL
Payment Demo ledger and payment adapter; payment_db	Model payment method, fee/deposit authorization, capture, void/refund, late-fee settlement and payout	Local ledger adapter; no real provider calls. Booking/Rental publish workflow notifications. Stable financial operation IDs prevent duplicates.	REST	PostgreSQL
Review and Reputation Participant reviews; review_db	Submit review; read reputation summary	Rental: verify CLOSED status and participation before accepting review. No review notification event in current scope.	REST	MongoDB
Notification Event delivery and inbox; notification_db	Consume event; upsert/list inbox; reconnect after broker failure	RabbitMQ durable queue → MongoDB inbox. Acknowledge after storage; deduplicate (eventId, recipient); dead-letter malformed events.	RabbitMQ; REST inbox	MongoDB

3.1 Deferred Capabilities and Design Rationale

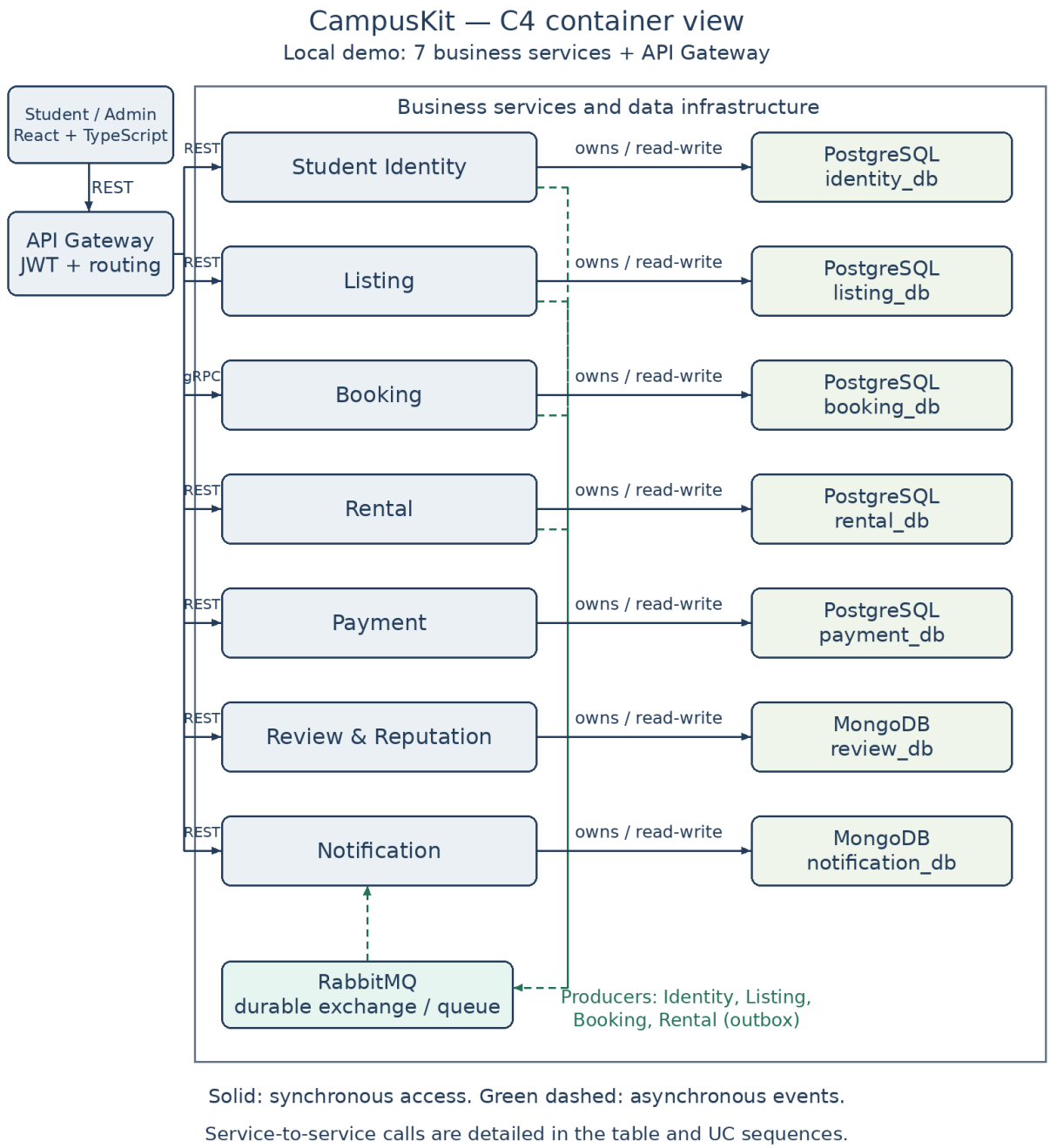
Identity account-restriction management, erasure, identity-document purge and pseudonymisation remain proposed operations. Booking calendar reconciliation is separate from the implemented request-expiry/compensation worker. Rental holdSettlement/dispute

workflows, Payment real-provider reconciliation, and Notification external contact/delivery retry are deferred. Damage/loss claims and dispute adjudication have no service or database in this iteration.

PostgreSQL supports transactional reservations, custody and financial records; Booking also needs range/exclusion constraints. MongoDB stores each review or inbox entry as a document and supports compound unique indexes for business-level deduplication. This is a suitable course choice, not a claim that relational storage could not handle these records. Two database technologies add index, credential, backup and recovery work. Shared local instances reduce setup cost but remain infrastructure failure points.

Seven services demonstrate independent ownership and deployment for the course; for a five-person team, they also increase contract testing and recovery work. One shared Compose configuration and a reduced feature scope keep that cost manageable (ADR-001).

4. Architecture Diagram (C4 Level 2: Containers)



The diagram shows current containers and database ownership. REST/gRPC calls are synchronous; outbox events are asynchronous. Detailed service calls appear in Section 3 and the UC sequences.

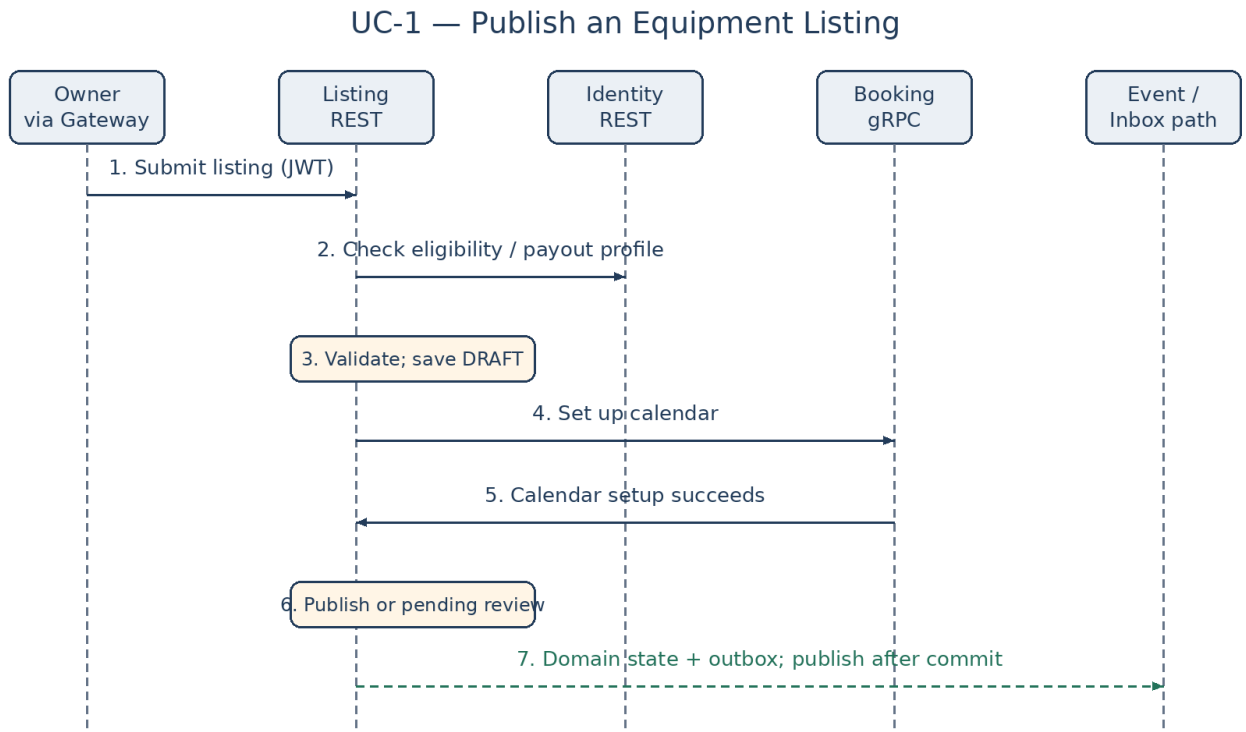
Production integrations are deferred: Payment → real payment provider; Identity → university IdP; Notification → email/push; relevant services → S3. They are outside the current

deployment diagram. The local payment adapter is part of Payment, not an external gateway. Detailed synchronous collaborations appear in Section 3 and the sequence diagrams below.

5. Sequence Diagrams

The diagrams group student/admin UI and API Gateway as the calling party. Protected requests carry JWTs; the data-owning service checks role, ownership or participation. Event/inbox lanes summarise the asynchronous path defined in ADR-008, not a synchronous Notification API call.

5.1 UC-1: Publish an Equipment Listing

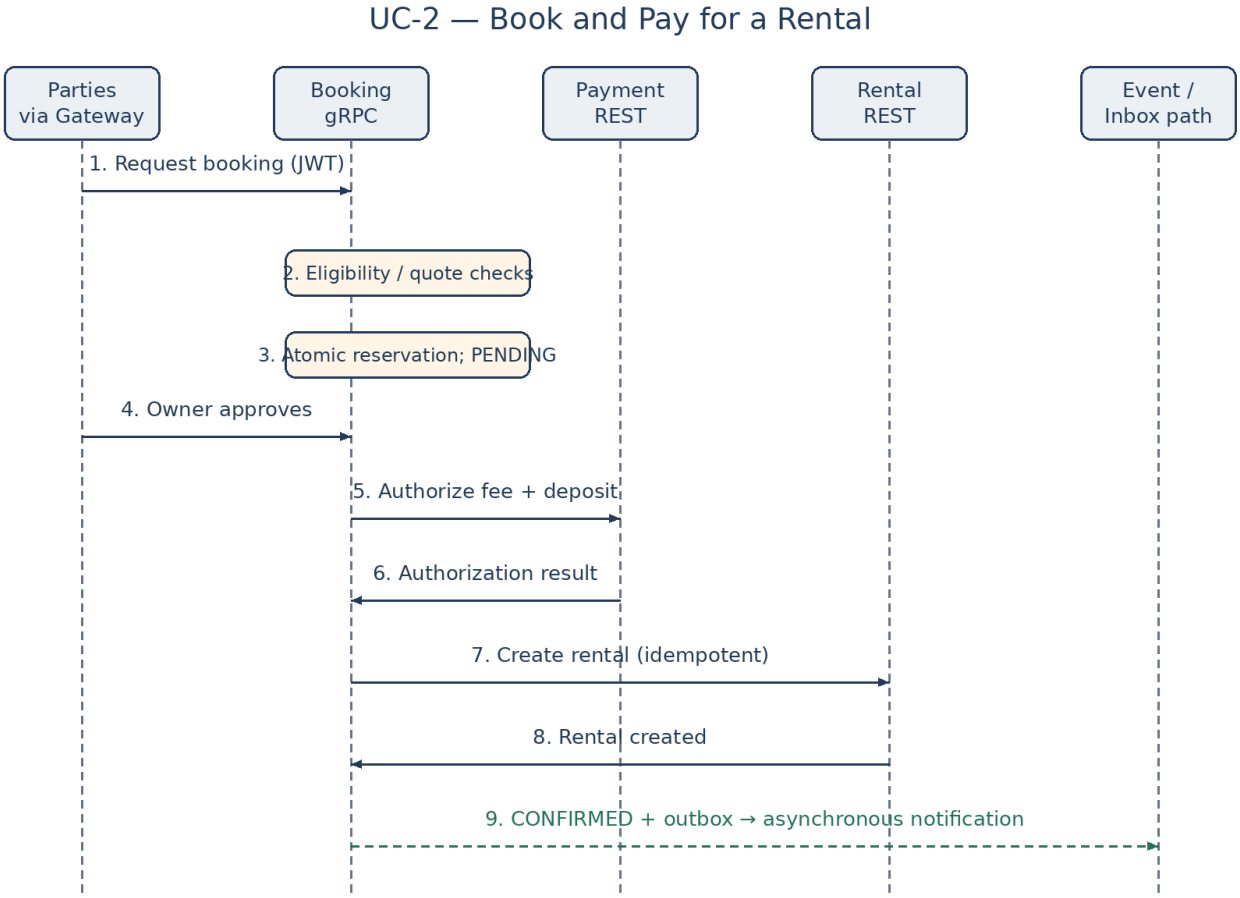


Event path: outbox → RabbitMQ → Notification → MongoDB inbox.

Moderated listings publish after approval; invalid input is rejected.

Calendar failure keeps the listing unpublished. Broker failure delays the inbox.

5.2 UC-2: Book and Pay for a Rental

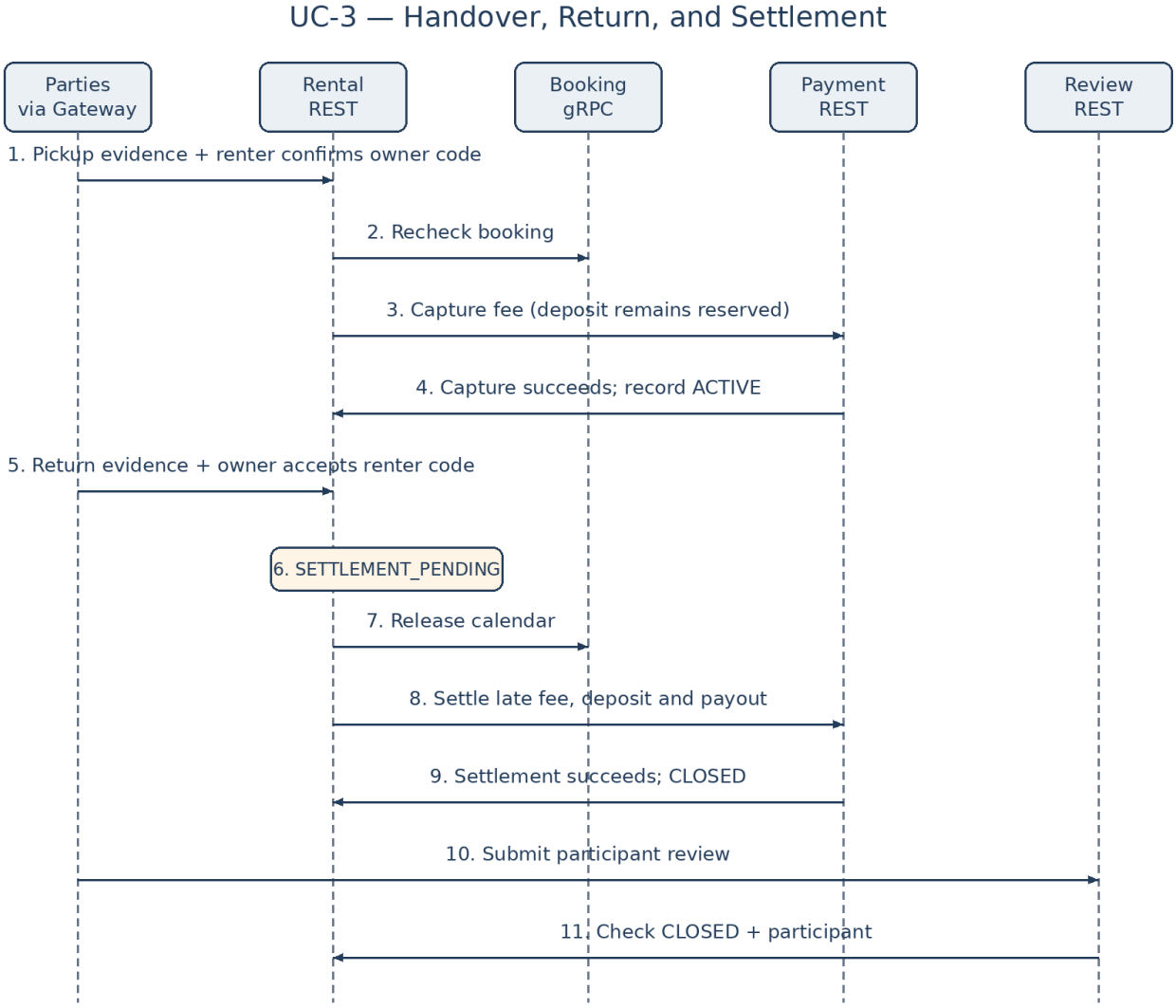


Step 2: Booking calls Identity and Listing through REST.

Failed authorization: retry within window; expiry releases dates and voids holds.

Failed confirmation: compensate payment / reservation; track unresolved recovery.

5.3 UC-3: Complete Handover, Return, and Settlement

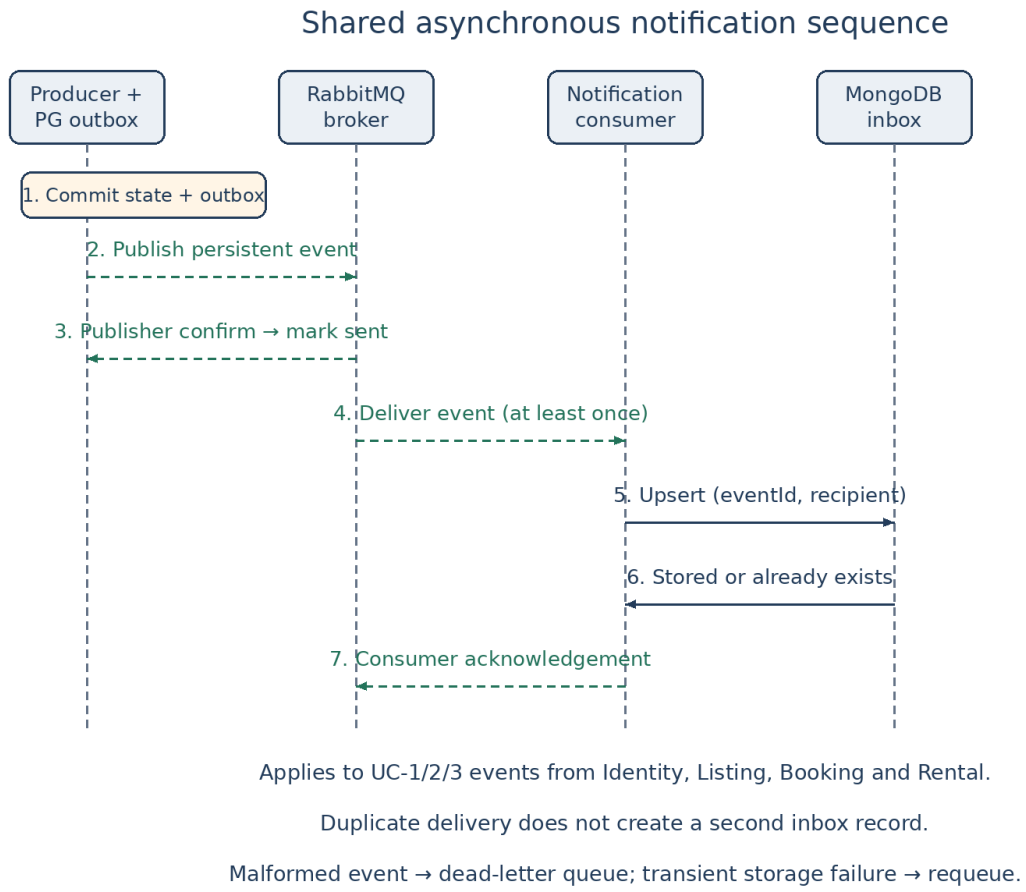


Rental domain commits also write outbox events (asynchronous).

Outbox → RabbitMQ → Notification → MongoDB inbox; no review notification event.

Settlement failure preserves custody and SETTLEMENT_PENDING for explicit retry.

5.4 Asynchronous Notification Delivery



Identity, Listing, Booking and Rental commit domain state and outbox events in the same PostgreSQL transaction. A publisher sends persistent events to RabbitMQ and marks an entry sent after publisher confirmation. Notification upserts MongoDB inbox records before acknowledging delivery; a unique (eventId, recipient) index deduplicates redelivery. Malformed events go to a dead-letter queue; transient storage failures are requeued. Broker outages delay the inbox without reversing committed business state. Review and Payment do not publish separate notification events in the current scope.

6. Service Discovery and Local Deployment

Docker Compose DNS resolves service names within the network. Services read endpoint addresses from environment configuration rather than hard-coding container IPs. The gateway translates client REST requests to Booking gRPC where needed; Listing and Rental also call Booking through gRPC.

Identity — identity:3001 (REST)

Listing — listing:3002 (REST)

Booking — booking:50051 (gRPC)

Rental — rental:3003 (REST)

Payment — payment:3004 (REST)

Review — review:3005 (REST)

Notification — notification:3006 (REST inbox; RabbitMQ consumer)

The local deployment uses one PostgreSQL instance and one MongoDB instance with separate logical databases and credentials per service, plus persistent volumes. There is no external service registry or advanced multi-instance gRPC balancing. Volumes alone do not prove backup/restore targets; RPO/RTO, TLS and load/usability testing remain acceptance or production targets as stated in Tab 2.

7. Collaboration Checks

UC-1: validate before saving DRAFT; calendar setup must succeed before publication. UC-2: prevent overlapping committed reservations; retry authorization within its window and compensate for failed confirmation. UC-3: retain the deposit after fee capture; accept evidence/code at return, release the calendar, settle, then close. Review requires a CLOSED rental and participant authorization. Verify broker outage/recovery and duplicate event delivery separately from payment workflow recovery. These checks are a validation plan, not claimed test results.